

Lab 5.1. Create XML layouts for Android

1. Before you begin

In this codelab, you will be building the layout for a basic tip calculator app. At the end of the codelab, you'll have a working UI for the app, but the app won't actually calculate the tip yet. Making the app work and look more professional will be in the following codelabs.

Prerequisites

- Able to create and run an Android app from a template in Android Studio

What you'll learn

- How to read and write XML layouts in Android
- How to build the layout for a simple form to take in user text input and choices

What you'll build

- The UI for a tip calculator Android app

What you need

- A computer with the latest stable version of Android Studio installed
- Internet connection to access the [Android developer documentation](https://developer.android.com/)

2. Start the project

Check out the tip calculator on Google: <https://www.google.com/search?q=tip+calculator>

Bill	0.00	Tip	\$0.00
Tip %	- 15% +	Total	\$0.00
Number of people	- 1 +		

In this pathway, you'll build a simple version of a tip calculator as an Android app.

Developers will often work in this way—getting a simple version of the app ready and partially functioning (even if it doesn't look very good), and then making it fully functional and visually polished later.

By the end of this codelab, your tip calculator app will look like this:

10:47

Tip Time

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

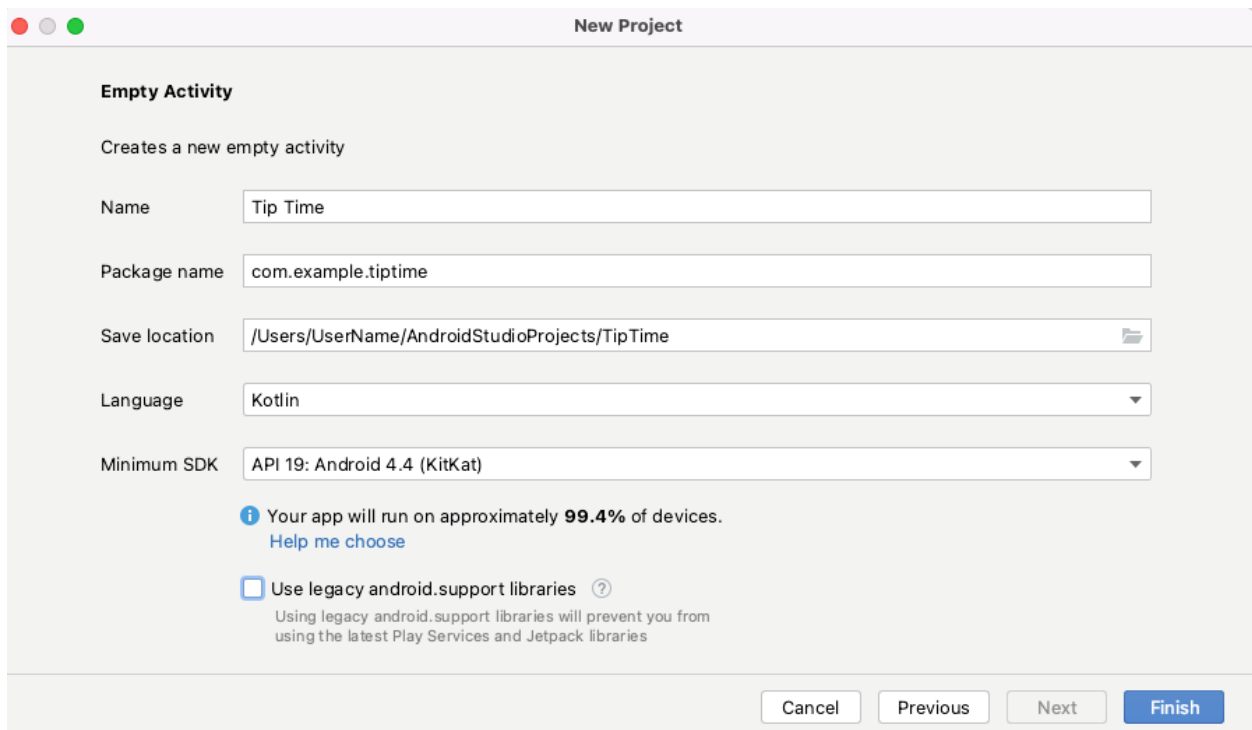
Tip Amount

You will be using these UI elements that are provided by Android:

- `EditText` - for entering and editing text
- `TextView` - to display text like the service question and tip amount
- `RadioButton` - a selectable radio button for each tip option
- `RadioGroup` - to group the radio button options
- `Switch` - an on/off toggle for choosing whether to round up the tip or not

Create an Empty Activity project

1. To start, create a new Kotlin project in Android Studio using the **Empty Activity** template.
2. Call the app "Tip Time", with a minimum API level of 19 (KitKat). The package name is **com.example.tiptime**.



3. Click **Finish** to create the app.

3. Read and understand XML

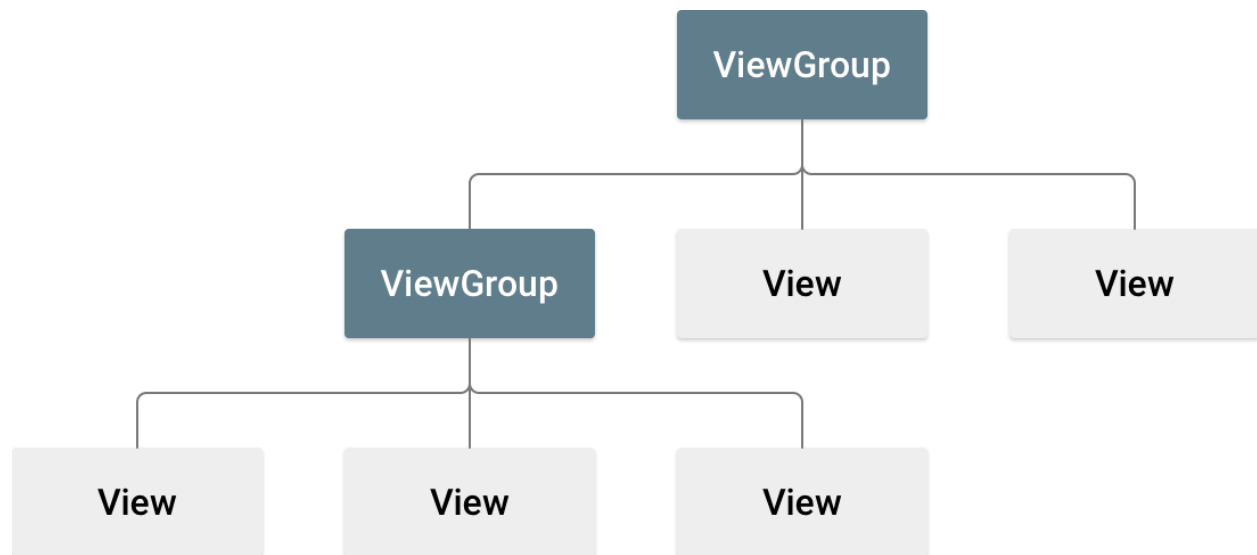
Instead of using the **Layout Editor** which you're already familiar with, you will build the layout of your application by modifying the [XML](#) that describes the UI. Learning how to understand and modify UI layouts using XML will be important for you as an Android developer.

You will be looking at, and editing the XML file that defines the UI layout for this app. XML stands for *eXtensible Markup Language*, which is a way of describing data using a text-based

document. Because XML is extensible and very flexible, it's used for many different things, including defining the UI layout of Android apps. You may recall from earlier codelabs that other resources like strings for your app are also defined in an XML file called `strings.xml`.

The UI for an Android app is built as a containment hierarchy of components (widgets), and the on-screen layouts of those components. Note those layouts are UI components themselves.

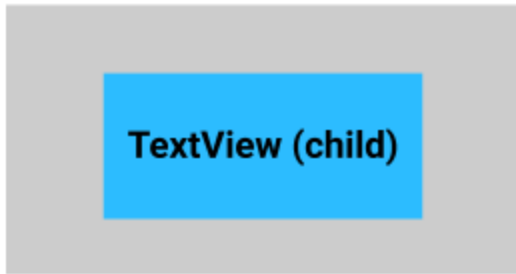
You describe the view hierarchy of UI elements on the screen. For example, a `ConstraintLayout` (the parent) can contain `Buttons`, `TextViews`, `ImageViews`, or other views (the children). Remember, `ConstraintLayout` is a subclass of `ViewGroup`. It allows you to position or size child views in a flexible manner.



Containment hierarchy of an Android app

NOTE: The visible UI hierarchy is based on containment, ie, one component has one or more components inside of it. This is unrelated to the hierarchy of classes and subclasses that you learned earlier. The terms parent and child are sometimes used, but the context here is talking about parent views (viewgroups) containing children views, which in turn can contain children views.

ConstraintLayout (parent)



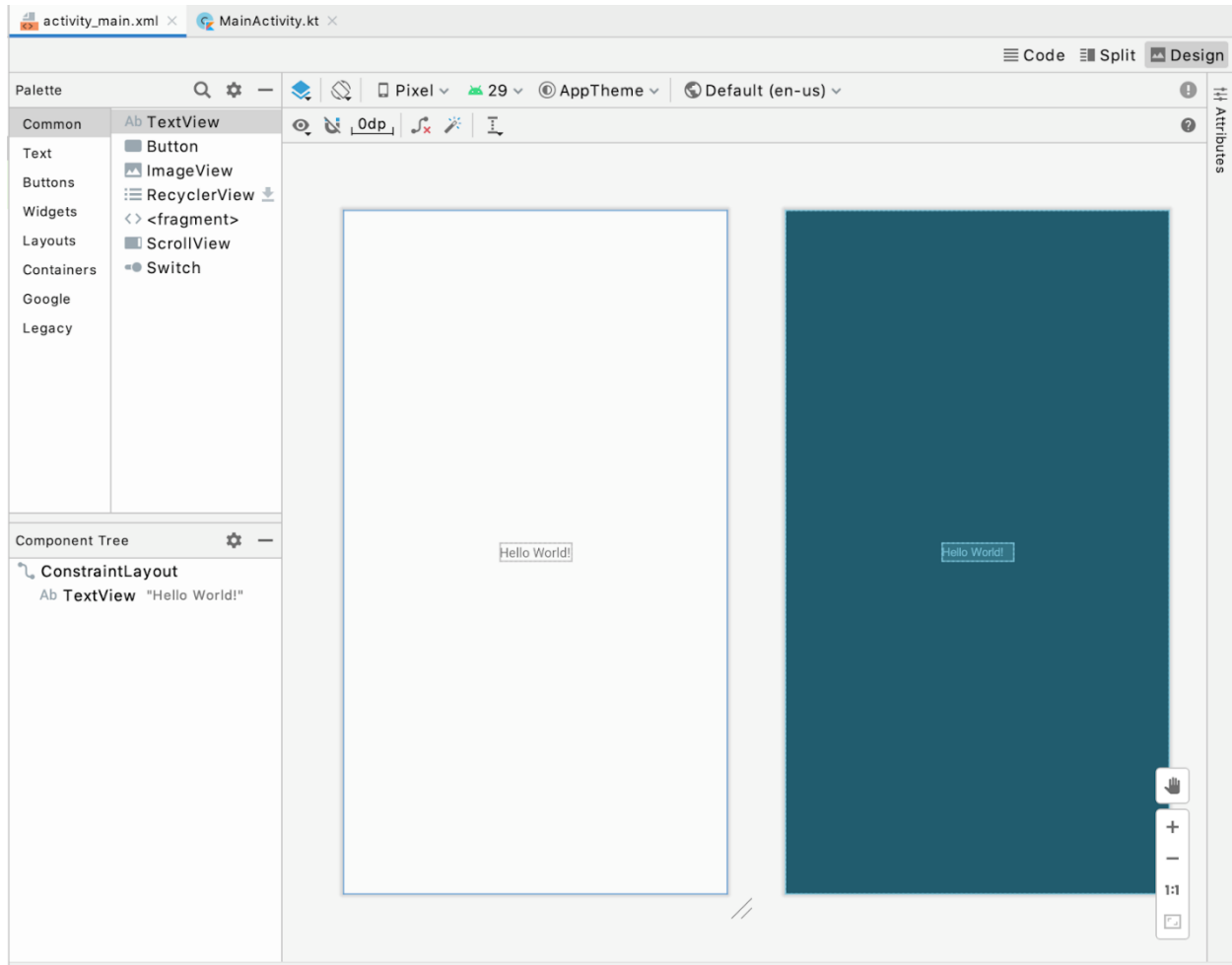
Each UI element is represented by an XML *element* in the XML file. Each element starts and ends with a tag, and each tag starts with a < and ends with a >. Just as you can set attributes on UI elements using the **Layout Editor (design)**, the XML elements can have *attributes*, too. Simplified, the XML for the UI elements above might be something like this:

```
<ConstraintLayout>
    <TextView
        text="Hello World!">
    </TextView>
</ConstraintLayout>
```

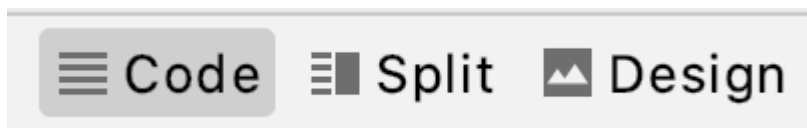


Let's look at a real example.

1. Open `activity_main.xml` (**res > layout > activity_main.xml**).
2. You might notice the app shows a `TextView` with "Hello World!" within a `ConstraintLayout`, as you have seen in previous projects created from this template.



3. Find the options for **Code**, **Split**, and **Design** views in the upper right of the Layout Editor.
4. Select the **Code** view.



This is what the XML in `activity_main.xml` looks like:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

There's a *lot* more going on than in the simplified example, but Android Studio does some things to help make the XML more readable, just as it does with your Kotlin code.

5. Notice the indentation. Android Studio does this automatically to show you the hierarchy of elements. The `TextView` is indented because it is contained in the `ConstraintLayout`. The `ConstraintLayout` is the parent, and the `TextView` is the child. The attributes for each element are indented to show that they're part of that element.
6. Notice the color coding—some things are in blue, some in green, and so on. Similar parts of the file are drawn in the same color to help you match them up. In particular, notice that Android Studio draws the start and end of tags of elements in the same color. (Note: the colors used in the codelab may not match what you see in Android Studio.)

XML tags, elements and attributes

Here's a simplified version of the `TextView` element so you can look at some of the important parts:

```

<TextView
    android:text="Hello World!"
/>

```

The line with `<TextView` is the start of the tag, and the line with `/>` is the end of the tag. The line with `android:text="Hello World!"` is an attribute of the tag. It represents text that will be displayed by the `TextView`. These 3 lines are a commonly used shorthand called an *empty-element tag*. It would mean the same thing if you wrote it with a separate *start-tag* and *end-tag*, like this:

```

<TextView
    android:text="Hello World!"
></TextView>

```

It's also common with an empty-element tag to write it on as few lines as possible and combine the end of the tag with the line before it. So you might see an empty-element tag on two lines (or even one line if it had no attributes):

```

<!-- with attributes, two lines -->
<TextView
    android:text="Hello World!" />

```

The `ConstraintLayout` element is written with separate start and end tags, because it needs to be able to hold other elements inside it. Here's a simplified version of the `ConstraintLayout` element with the `TextView` element inside it:

```
<androidx.constraintlayout.widget.ConstraintLayout>
    <TextView
        android:text="Hello World!" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

If you wanted to add another `View` as a child of the `ConstraintLayout`, like a `Button` below the `TextView`, it would go after the end of the `TextView` tag `/>` and before the end tag of the `ConstraintLayout`, like this:

```
<androidx.constraintlayout.widget.ConstraintLayout>
    <TextView
        android:text="Hello World!" />
    <Button
        android:text="Calculate" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

More about XML for layouts

1. Look at the tag for the `ConstraintLayout`, and notice that it says `androidx.constraintlayout.widget.ConstraintLayout` instead of just `ConstraintLayout` like the `TextView`. This is because `ConstraintLayout` is part of Android Jetpack, which contains libraries of code which offers additional functionality on top of the core Android platform. Jetpack has useful functionality you can take advantage of to make building apps easier. You'll recognize this UI component is part of Jetpack because it starts with "androidx".
2. You may have noticed the lines that begin with `xmlns:`, followed by `android`, `app`, and `tools`.

```
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
```

The `xmlns` stands for XML namespace, and each line defines a *schema*, or vocabulary for attributes related to those words. The `android:` namespace, for example, marks attributes that are defined by the Android system. All of the attributes in the layout XML begins with one of those namespaces.

3. Whitespace between XML elements doesn't change the meaning to a computer, but it can help make the XML easier for people to read.

Android Studio will automatically add some whitespace and indenting for readability. You'll learn later how to have Android Studio make sure your XML follows coding style conventions.

4. You can add comments to XML, just like you would with Kotlin code. Start `<!--` and end with `-->`.

```
<!-- this is a comment in XML -->
```

```
<!-- this is a
multi-line
Comment.
And another
Multi-line comment -->
```

5. Note the first line of the file:

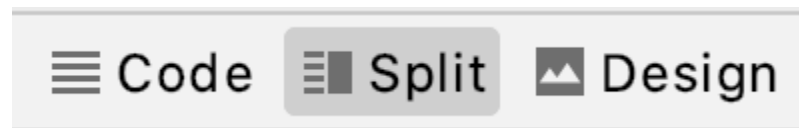
```
<?xml version="1.0" encoding="utf-8"?>
```

This indicates that the file is an XML file, but not every XML file includes this.

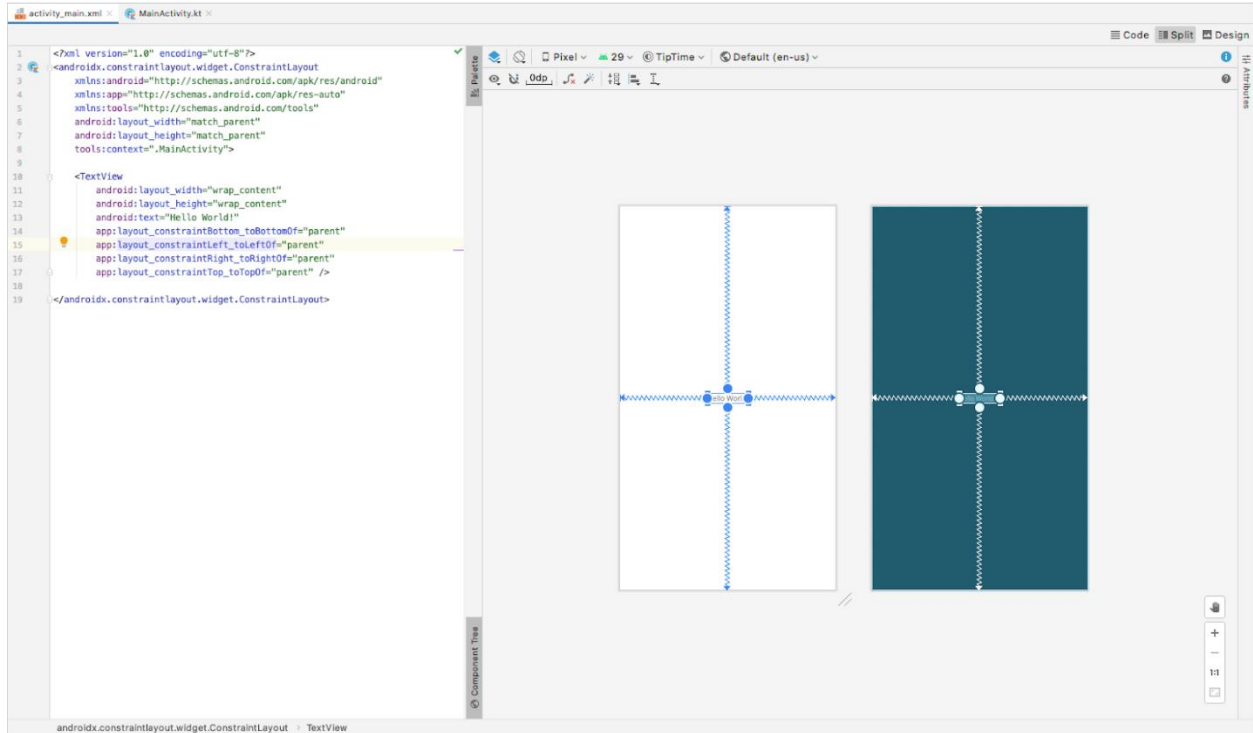
Note: If there's a problem with the XML for your app, Android Studio will flag the problem by drawing the text in red. If you move the mouse over the red text, Android Studio will show more information about the problem. If the problem isn't obvious, look at the indenting and color coding, and they may give you a clue to what is wrong.

4. Build the layout in XML

1. Still in `activity_main.xml`, switch to the **Split** screen view to see the XML beside the **Design Editor**. The **Design Editor** allows you to preview your UI layout.



2. It is up to personal preference which view you use, but for this codelab, use the **Split** view so you can see both the XML that you edit and the changes those edits make in the **Design Editor**.
3. Try clicking on different lines, one below the `ConstraintLayout`, and then one below the `TextView`, and notice that the corresponding view is selected in the **Design Editor**. The reverse works, too—for example, if you click on the `TextView` in the **Design Editor**, the corresponding XML is highlighted.



Delete the TextView

1. You don't need the `TextView` now, so delete it. Be sure to delete everything from the `<TextView` to the closing `</>`.

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

All that's left in the file is the `ConstraintLayout`:

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

</androidx.constraintlayout.widget.ConstraintLayout>
```

2. Add 16dp of padding to the `ConstraintLayout` so the UI won't be crowded against the edge of the screen.

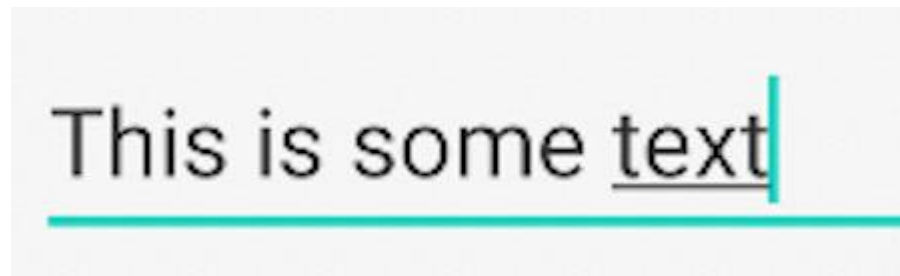
Padding is similar to margins, but it adds space to the inside of the `ConstraintLayout`, instead of adding space to the outside.

```
<androidx.constraintlayout.widget.ConstraintLayout
    ...
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".MainActivity">
```

Note: Some code snippets in this codelab won't show everything for brevity. Code that hasn't changed or isn't relevant for the current step will be represented with an ellipsis (3 consecutive dots ...), so you can focus on the most important parts of the code.

Add cost of service text field

In this step you'll add the UI element to allow the user to enter the cost of service into the app. You'll use an `EditText` element, which lets a user enter or modify text in an app.



1. Look at the [EditText](#) documentation, and examine the sample XML.
2. Find a blank space between the opening and closing tags of the `ConstraintLayout`.
3. Copy and paste the XML from the documentation into that space in your layout in Android Studio.

Your layout file should look like this:

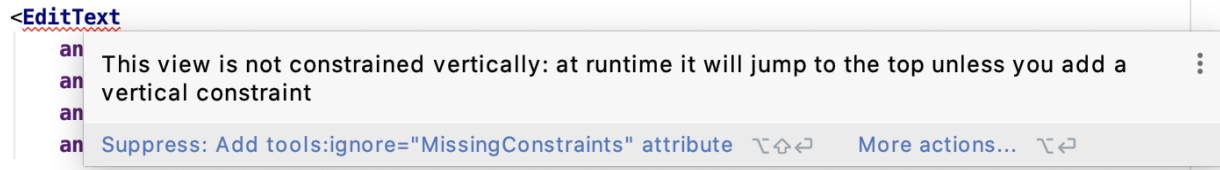
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/plain_text_input"
        android:layout_height="wrap_content"
        android:layout_width="match_parent"
        android:inputType="text"/>
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

You may not understand all of this yet, but it will be explained in the following steps.

4. Notice `EditText` is underlined in red.
5. Hover the pointer over it, and you'll see a "view is not constrained" error, which should look familiar from earlier codelabs. Recall that children of a `ConstraintLayout` need constraints so the layout knows how to arrange them.



6. Add these constraints to the `EditText` to anchor it to the top left corner of the parent.

```
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent"
```

If you're writing in English or another language that's written left-to-right (LTR), the starting edge is the left. But some languages like Arabic are written right-to-left (RTL), so the starting edge is the right. That's why the constraint uses "start", so that it can work with either LTR or RTL languages. Similarly, constraints use "end" instead of right.

Note: The name of the constraints follows the form

`layout_constraint<Source>_to<Target>Of`, where `<Source>` refers to the current view. `<Target>` refers to the edge of the target view that the current view is being constrained to, either the parent container or another view.

With the new constraints added, the `EditText` element will look like this:

```
<EditText
    android:id="@+id/plain_text_input"
    android:layout_height="wrap_content"
    android:layout_width="match_parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:inputType="text"/>
```

Review the `EditText` attributes

Double check all the `EditText` attributes that you pasted in to make sure it works for how it will be used in your app.

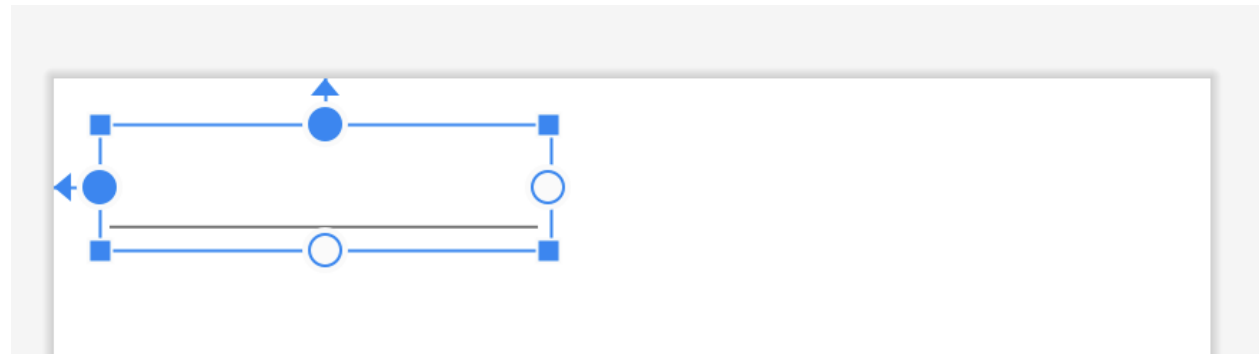
1. Find the `id` attribute, which is set to `@+id/plain_text_input`.
2. Change the `id` attribute to a more appropriate name, `@+id/cost_of_service`.

Note: A resource ID is a unique resource name for the element. When you add a `View` or other resource with the **Layout Editor**, Android Studio automatically assigns resource IDs for them. When you manually type out the XML, you need to explicitly declare the resource ID yourself. New view IDs in your XML file must be defined with the `@+id` prefix, which tells Android Studio to add that ID as a new resource ID.

Choose descriptive names for resources so you know what they refer to, but they should be all in lowercase letters, and multiple words should be separated with an underscore.

When you refer to resource IDs in your app code, use `R.<type>.<name>`; for example, `R.string.roll`. For View IDs, the `<type>` is `id`, for example, `R.id.button`.

3. Look at the `layout_height` attribute. It's set to `wrap_content` which means the height will be as tall as the content inside it. That's OK, because there will only be 1 line of text.
4. Look at the `layout_width` attribute. It's set to `match_parent`, but you can't set `match_parent` on a child of `ConstraintLayout`. Furthermore, the text field doesn't need to be so wide. Set it to be a fixed width of `160dp`, which should be plenty of space for the user to enter in a cost of service.



5. Notice the `inputType` attribute—it's something new. The value of the attribute is `"text"`, which means the user can type in any text characters into the field on screen (alphabetical characters, symbols, etc.)

```
android:inputType="text"
```

However, you want them to only type numbers into the `EditText`, because the field represents a monetary value.

6. Erase the word `text`, but leave the quotes.
7. Start typing `number` in its place. After typing `"n"`, Android Studio shows a list of possible completions that include `"n"`.

```

    android:inputType="text" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

8. Choose `numberDecimal`, which limits them to numbers with a decimal point.

```
android:inputType="numberDecimal"
```

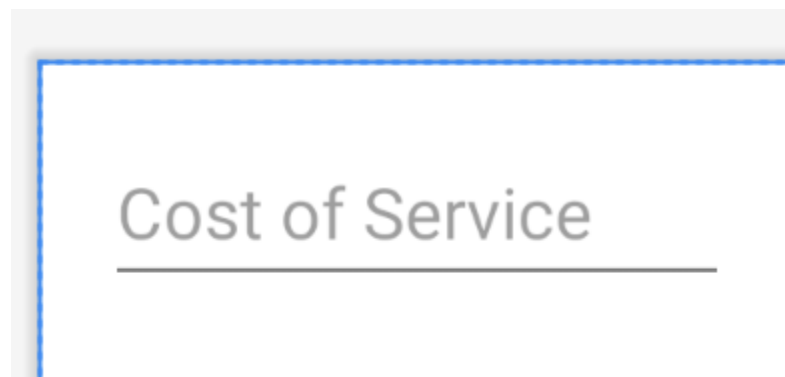
To see other options for input types, see [Specify the input method type](#) in the developer documentation.

There's one more change to make, because it's helpful to display some hint as to what the user should enter into this field.

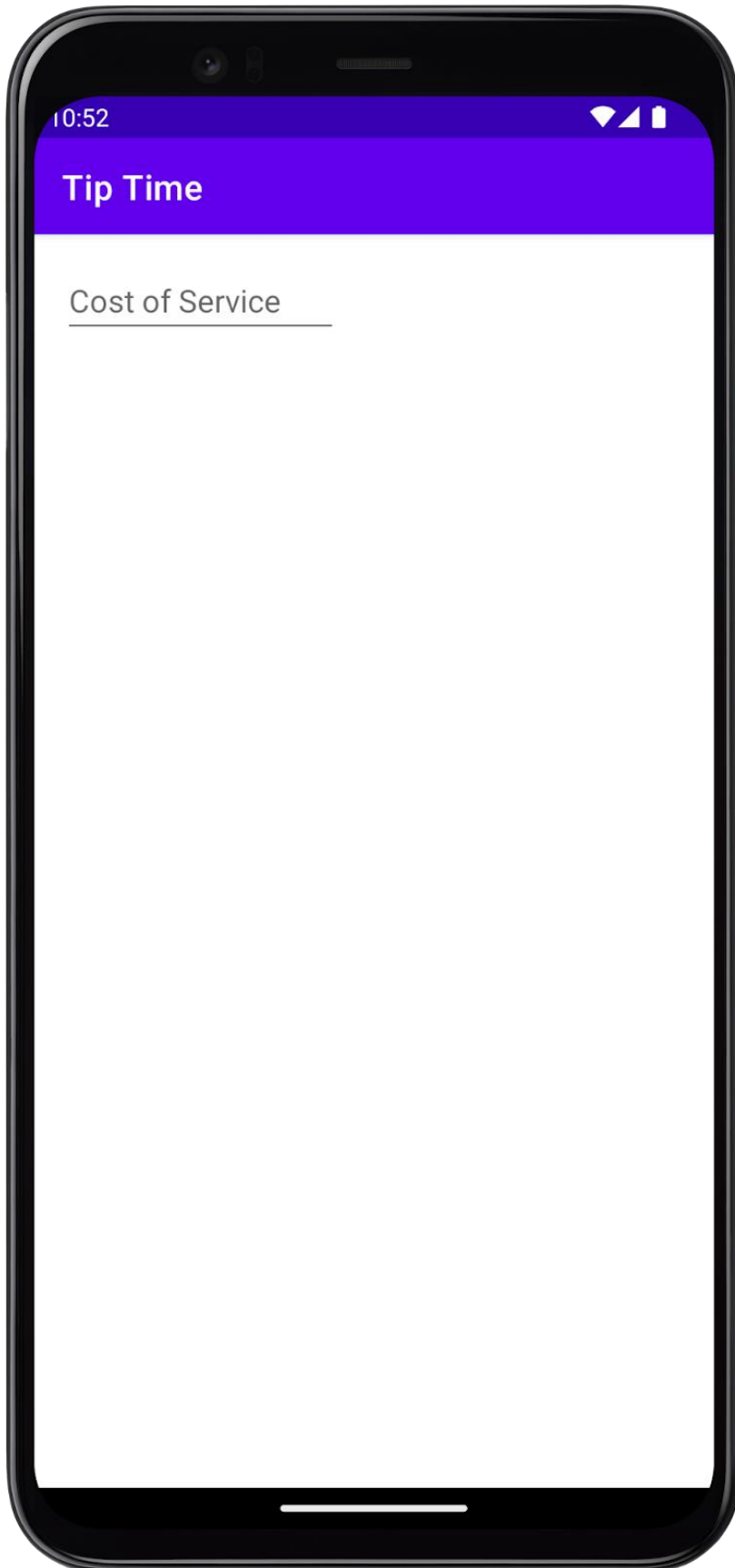
9. Add a `hint` attribute to the `EditText` describing what the user should enter in the field.

```
android:hint="Cost of Service"
```

You will see the **Design Editor** update, too.



10. Run your app in the emulator. It should look like this:



Nice job! It doesn't do much yet, but you've got a good start and have edited some XML. The XML for your layout should look something like this.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".MainActivity">

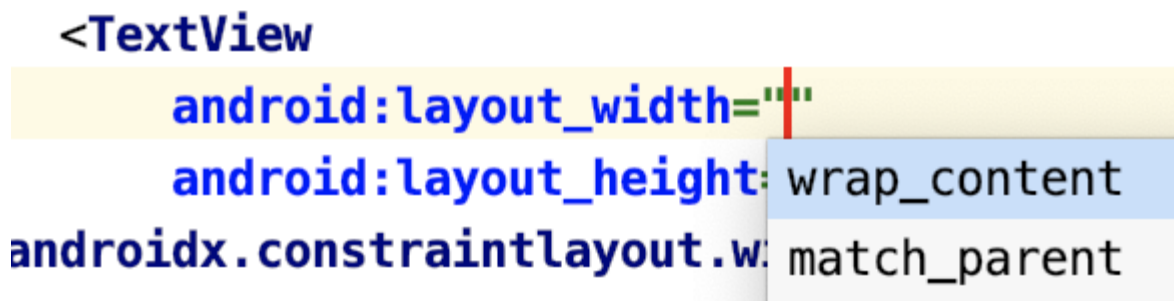
    <EditText
        android:id="@+id/cost_of_service"
        android:layout_width="160dp"
        android:layout_height="wrap_content"
        android:hint="Cost of Service"
        android:inputType="numberDecimal"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Add the service question

In this step, you'll add a `TextView` for the question prompt, "How was the service?" Try typing this out without copy/pasting. The suggestions from Android Studio should help you.

1. After the close of the `EditText` tag, `/>`, add a new line and start typing `<TextView`.
2. Select `TextView` from the suggestions, and Android Studio will automatically add the `layout_width` and `layout_height` attributes for the `TextView`.
3. Choose `wrap_content` for both, since you only need the `TextView` to be as big as the text content inside it.

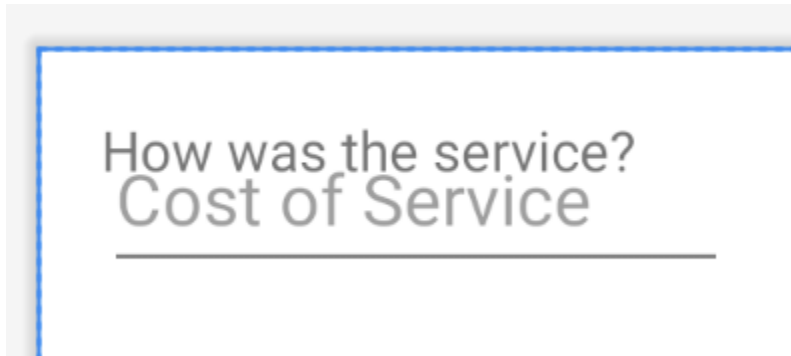


4. Add the `text` attribute with "How was the service?"

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="How was the service?"
```

5. Close the tag with `/>`.

6. Notice in the **Design Editor** that the `TextView` overlaps the `EditText`.



That doesn't look right, so you'll add constraints on the `TextView` next. Think about what constraints you need. Where should the `TextView` be positioned horizontally and vertically? You can check the app screenshot to help you.

10:47

Tip Time

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

Tip Amount

Vertically, you want the `TextView` to be below the cost of service text field. Horizontally, you want the `TextView` aligned to the starting edge of the parent.

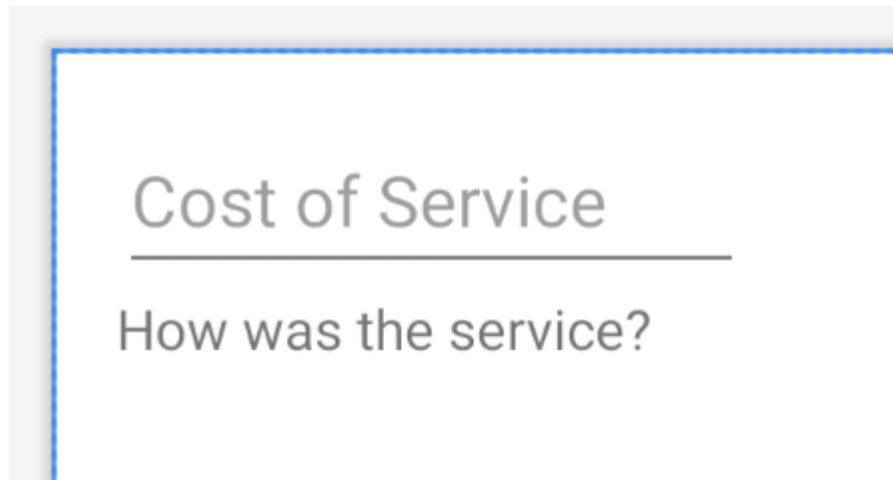
7. Add a horizontal constraint to the `TextView` to constrain its starting edge to the starting edge of the parent.

```
app:layout_constraintStart_toStartOf="parent"
```

8. Add a vertical constraint to the `TextView` to constrain the top edge of the `TextView` to the bottom edge of the cost of service `View`.

```
app:layout_constraintTop_toBottomOf="@id/cost_of_service"
```

Note that there's no plus in `@id/cost_of_service` because the ID is already defined.



It doesn't look the best, but don't worry about that for now. You just want to make sure all the necessary pieces are on screen and the functionality works. You'll fix how it looks in the following codelabs.

9. Add a resource ID on the `TextView`. You'll need to refer to this view later as you add more views and constrain them to each other.

```
android:id="@+id/service_question"
```

At this point, your XML should look like this.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">
```

```

tools:context=".MainActivity">

<EditText
    android:id="@+id/cost_of_service"
    android:hint="Cost of Service"
    android:layout_height="wrap_content"
    android:layout_width="160dp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:inputType="numberDecimal"/>

<TextView
    android:id="@+id/service_question"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="How was the service?"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/cost_of_service"/>

</androidx.constraintlayout.widget.ConstraintLayout>

```

5. Add tip options

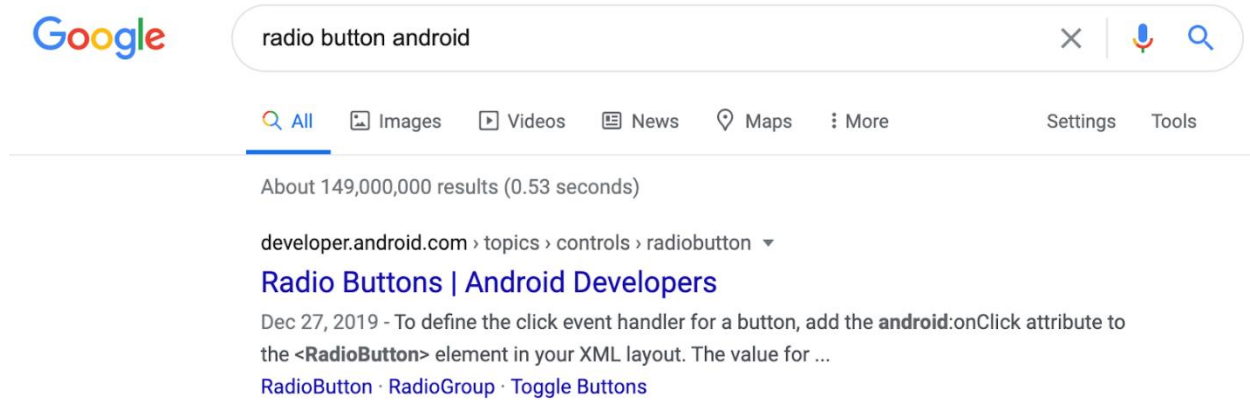
Next you'll add radio buttons for the different tip options the user can choose from.

There should be three options:

- Amazing (20%)
- Good (18%)
- Okay (15%)

If you're not sure how to do this, you can do a Google search. This is a great tool that developers use when they're stuck.

1. Do a Google search for `radio button android`. The top result is a guide from the Android developers site on how to use radio buttons—perfect!

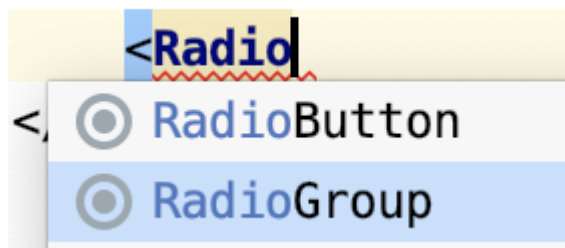


2. Skim through the [Radio Buttons guide](#).

Reading the description, you can confirm that you can use a `RadioButton` UI element in your layout for each radio button you need. Furthermore, you also need to group the radio buttons within a `RadioGroup` because only one option can be selected at a time.

There is some XML that looks like it would fit your needs. Read through it and see how `RadioGroup` is the parent view and the `RadioButtons` are child views within it.

3. Go back to your layout in Android Studio to add the `RadioGroup` and `RadioButton` to your app.
4. After the `TextView` element, but still within the `ConstraintLayout`, start typing out `<RadioGroup`. Android Studio will provide helpful suggestions to help you complete



your XML.

5. Set the `layout_width` and `layout_height` of the `RadioGroup` to `wrap_content`.
6. Add a resource ID set to `@+id/tip_options`.
7. Close the start tag with `>`.

- Android Studio adds `</RadioGroup>`. Like the `ConstraintLayout`, the `RadioGroup` element will have other elements inside it, so you might want to move it to its own line.

```
<TextView
    android:id="@+id/service_question"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="How was the service?"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/cost_of_service" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

- Constrain the `RadioGroup` below the service question (vertically) and to the start of the parent (horizontally).
- Set the `android:orientation` attribute to `vertical`. If you wanted the `RadioButtons` in a row, you would set the orientation to `horizontal`.

The XML for the `RadioGroup` should look like this:

```
<RadioGroup
    android:id="@+id/tip_options"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/service_question">

</RadioGroup>
```

Add RadioButtons

- After the last attribute of the `RadioGroup`, but before the `</RadioGroup>` end tag, add a `RadioButton`.

```
<RadioGroup
    android:id="@+id/tip_options"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/service_question">

    <!-- add RadioButtons here -->

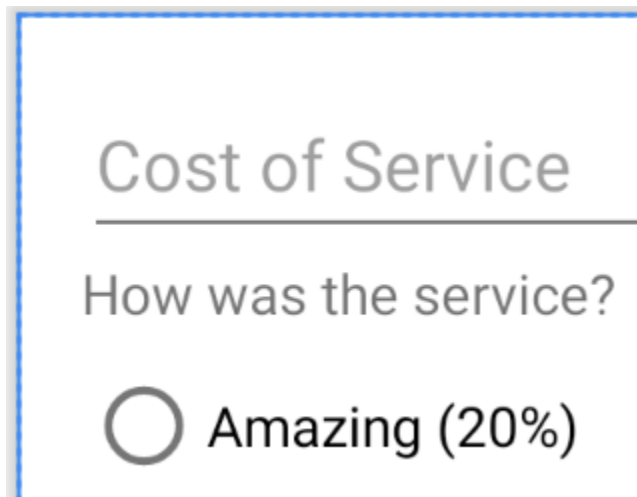
</RadioGroup>
```

2. Set the `layout_width` and `layout_height` to `wrap_content`.
3. Assign a resource ID of `@+id/option_twenty_percent` to the `RadioButton`.
4. Set the text to `Amazing (20%)`.
5. Close the tag with `/>`.

```
<RadioGroup
    android:id="@+id/tip_options"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintTop_toBottomOf="@id/service_question"
    app:layout_constraintStart_toStartOf="parent"
    android:orientation="vertical">

    <RadioButton
        android:id="@+id/option_twenty_percent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Amazing (20%)" />

</RadioGroup>
```



Now that you've added one `RadioButton`, can you modify the XML to add 2 more radio buttons for the `Good (18%)` and `Okay (15%)` options?

This is what the XML for the `RadioGroup` and `RadioButtons` looks like:

```
<RadioGroup
    android:id="@+id/tip_options"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintTop_toBottomOf="@id/service_question"
    app:layout_constraintStart_toStartOf="parent"
    android:orientation="vertical">

    <RadioButton
        android:id="@+id/option_twenty_percent"
```

```

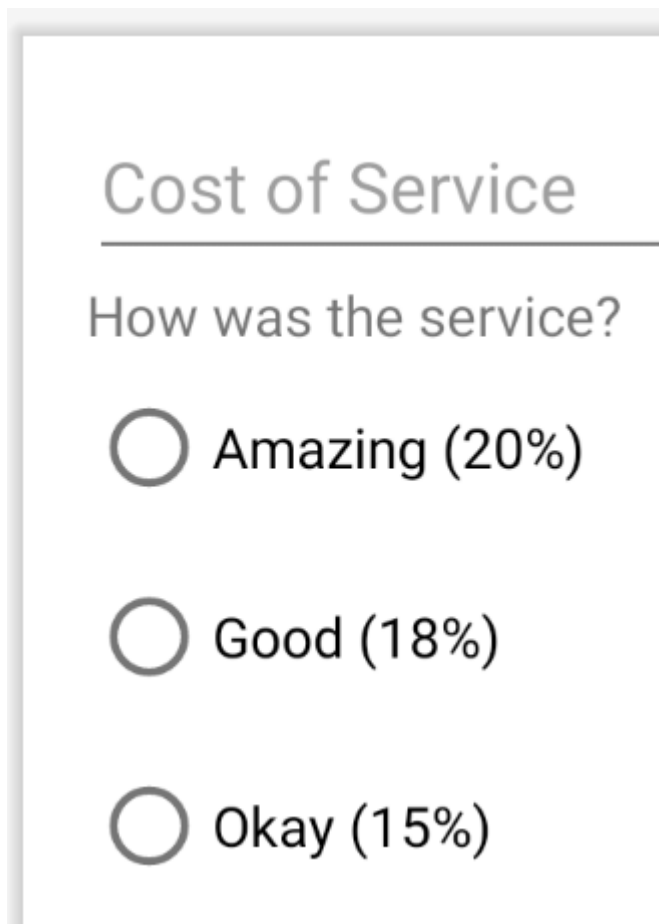
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Amazing (20%) " />

<RadioButton
    android:id="@+id/option_eighteen_percent"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Good (18%) " />

<RadioButton
    android:id="@+id/option_fifteen_percent"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Okay (15%) " />

</RadioGroup>

```



Cost of Service

How was the service?

☐ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Add a default selection

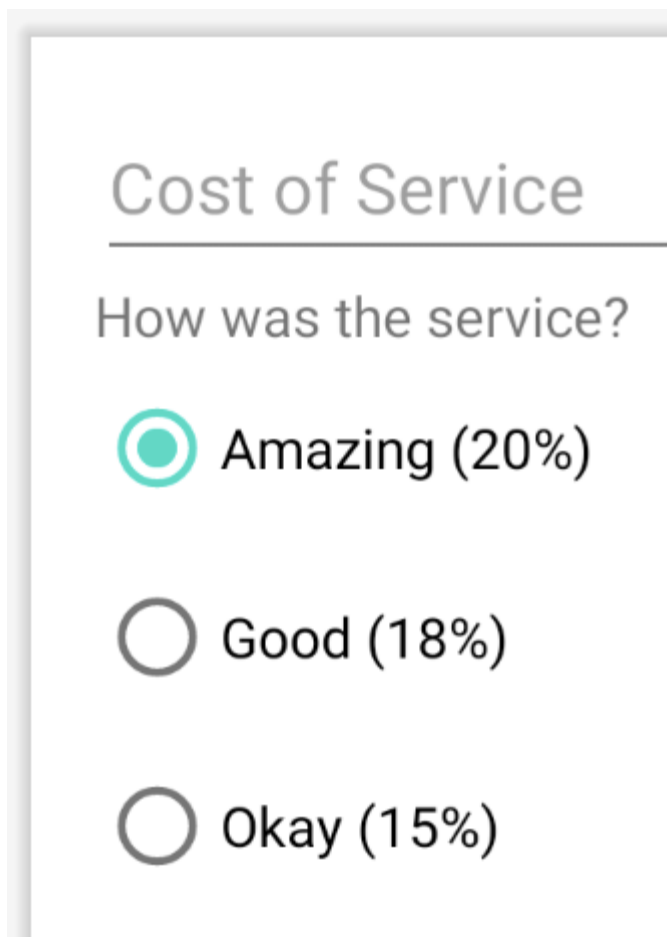
Currently, none of the tip options are selected. It would be nice to select one of the radio button options by default.

There's an attribute on the `RadioGroup` where you can specify which button should be checked initially. It's called `checkedButton`, and you set it to the resource ID of the radio button you want selected.

1. On the `RadioGroup`, set the `android:checkedButton` attribute to `@id/option_twenty_percent`.

```
<RadioGroup
    android:id="@+id/tip_options"
    android:checkedButton="@id/option_twenty_percent"
    ...
```

Notice in the **Design Editor** that the layout has been updated. The 20% tip option is selected by default—cool! Now it's starting to look like a tip calculator!



Here is what the XML looks like so far:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
```

```

        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp"
        tools:context=".MainActivity">

        <EditText
            android:id="@+id/cost_of_service"
            android:hint="Cost of Service"
            android:layout_height="wrap_content"
            android:layout_width="160dp"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            android:inputType="numberDecimal"/>

        <TextView
            android:id="@+id/service_question"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="How was the service?"
            app:layout_constraintTop_toBottomOf="@id/cost_of_service"
            app:layout_constraintStart_toStartOf="parent" />

        <RadioGroup
            android:id="@+id/tip_options"
            android:checkedButton="@id/option_twenty_percent"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            app:layout_constraintTop_toBottomOf="@id/service_question"
            app:layout_constraintStart_toStartOf="parent"
            android:orientation="vertical">

            <RadioButton
                android:id="@+id/option_twenty_percent"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Amazing (20%" />

            <RadioButton
                android:id="@+id/option_eighteen_percent"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Good (18%" />

            <RadioButton
                android:id="@+id/option_fifteen_percent"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Okay (15%" />
        </RadioGroup>
    </androidx.constraintlayout.widget.ConstraintLayout>

```

6. Complete the rest of the layout

You're on the last part of the layout now. You'll add a `Switch`, `Button`, and a `TextView` to display the tip amount.

10:47

Tip Time

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

Tip Amount

Add a Switch for rounding up the tip

Next, you'll use a [Switch](#) widget to allow the user to select yes or no for rounding up the tip.

You want the `Switch` to be as wide as the parent, so you might think the width should be set to `match_parent`. As noted earlier, you can't set `match_parent` on UI elements in a `ConstraintLayout`. Instead, you need to constrain the start and end edges of the view, and set the width to `0dp`. Setting the width to `0dp` tells the system not to calculate the width, just try to match the constraints that are on the view.

Note: You cannot use `match_parent` for any view in a `ConstraintLayout`. Instead use `0dp`, which means match constraints.

1. Add a `Switch` element after the XML for the `RadioGroup`.
2. As noted above, set the `layout_width` to `0dp`.
3. Set the `layout_height` to `wrap_content`. This will make the `Switch` view as tall as the content inside.
4. Set the `id` attribute to `@+id/round_up_switch`.
5. Set the `text` attribute to `Round up tip?`. This will be used as a label for the `Switch`.
6. Constrain the start edge of the `Switch` to the start edge of the `tip_options` and the end to the end of the parent.
7. Constrain the top of the `Switch` to the bottom of the `tip_options`.
8. Close the tag with `/>`.

It would be nice if the switch was turned on by default, and there's an attribute for that, `android:checked`, where the possible values are `true` (on) or `false` (off).

9. Set the `android:checked` attribute to `true`.

Putting that all together, the XML for the `Switch` element looks like this:

```
<Switch
    android:id="@+id/round_up_switch"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:checked="true"
    android:text="Round up tip?"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="@id/tip_options"
    app:layout_constraintTop_toBottomOf="@id/tip_options" />
```

Note: If you hover over the `Switch` element in your XML layout, you will see a suggestion in Android Studio that says: "Use `SwitchCompat` from `AppCompat` or `SwitchMaterial` from `Material` library." You will implement this suggestion in a later codelab of building the tip calculator app, so you can ignore the warning in the meantime.

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

Add the Calculate button

Next you'll add a `Button` so the user can request that the tip be calculated. You want the button to be as wide as the parent, so the horizontal constraints and width are the same as they were for the `Switch`.

1. Add a `Button` after the `Switch`.
2. Set the width to `0dp`, as you did for the `Switch`.
3. Set the height to `wrap_content`.
4. Give it a resource ID of `@+id/calculate_button`, with the text "Calculate".
5. Constrain the top edge of `Button` to the bottom edge of the **Round up tip?** `Switch`.
6. Constrain the start edge to the start edge of the parent and the end edge to the end edge of the parent.
7. Close the tag with `/>`.

Here's what the XML for the **Calculate** `Button` looks like:

```
<Button
    android:id="@+id/calculate_button"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="Calculate"
    app:layout_constraintTop_toBottomOf="@id/round_up_switch"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

Add tip result

You're almost done with the layout! In this step you'll add a `TextView` for the tip result, putting it below the **Calculate** button, and aligned with the end instead of the start like the other UI elements.

1. Add a `TextView` with a resource ID named `tip_result` and the text `Tip Amount`.
2. Constrain the ending edge of the `TextView` to the ending edge of the parent.
3. Constrain the top edge to the bottom edge of the **Calculate** button.

```
<TextView
    android:id="@+id/tip_result"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@id/calculate_button"
    android:text="Tip Amount" />
```

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

Tip Amount

4. Run the app. It should look like this screenshot.

10:49

Tip Time

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

Great job, especially if this is your first time working with XML!

Note that the app may not look exactly the same as the screenshot because the templates may have changed in a later version of Android Studio. The **Calculate** button doesn't do anything yet, but you can type in the cost, select the tip percentage, and toggle the option to round up the tip or not. You'll make the **Calculate** button work in the next codelab, so be sure to come back for it!

7. Adopt good coding practices

Extract the strings

You may have noticed the warnings about hard-coded strings. Recall from the earlier codelabs that extracting strings to a resource file makes it easier to translate your app to other languages and to reuse strings. Go through `activity_main.xml` and extract all the string resources.

1. Click on a string; hover over on the yellow light bulb icon that appears, then click on the triangle next to it; choose **Extract String Resource**. The default names for the string resources are fine. If you want, for the tip options you can use `amazing_service`, `good_service`, and `ok_service` to make the names more descriptive.

Now verify the string resources you just added.

2. If the **Project** window isn't showing, click the **Project** tab on the left side of the window.
3. Open **app > res > values > strings.xml** to see all the UI string resources.

```
<resources>
    <string name="app_name">Tip Time</string>
    <string name="cost_of_service">Cost of Service</string>
    <string name="how_was_the_service">How was the service?</string>
    <string name="amazing_service">Amazing (20%)</string>
    <string name="good_service">Good (18%)</string>
    <string name="ok_service">Okay (15%)</string>
    <string name="round_up_tip">Round up tip?</string>
    <string name="calculate">Calculate</string>
    <string name="tip_amount">Tip Amount</string>
</resources>
```

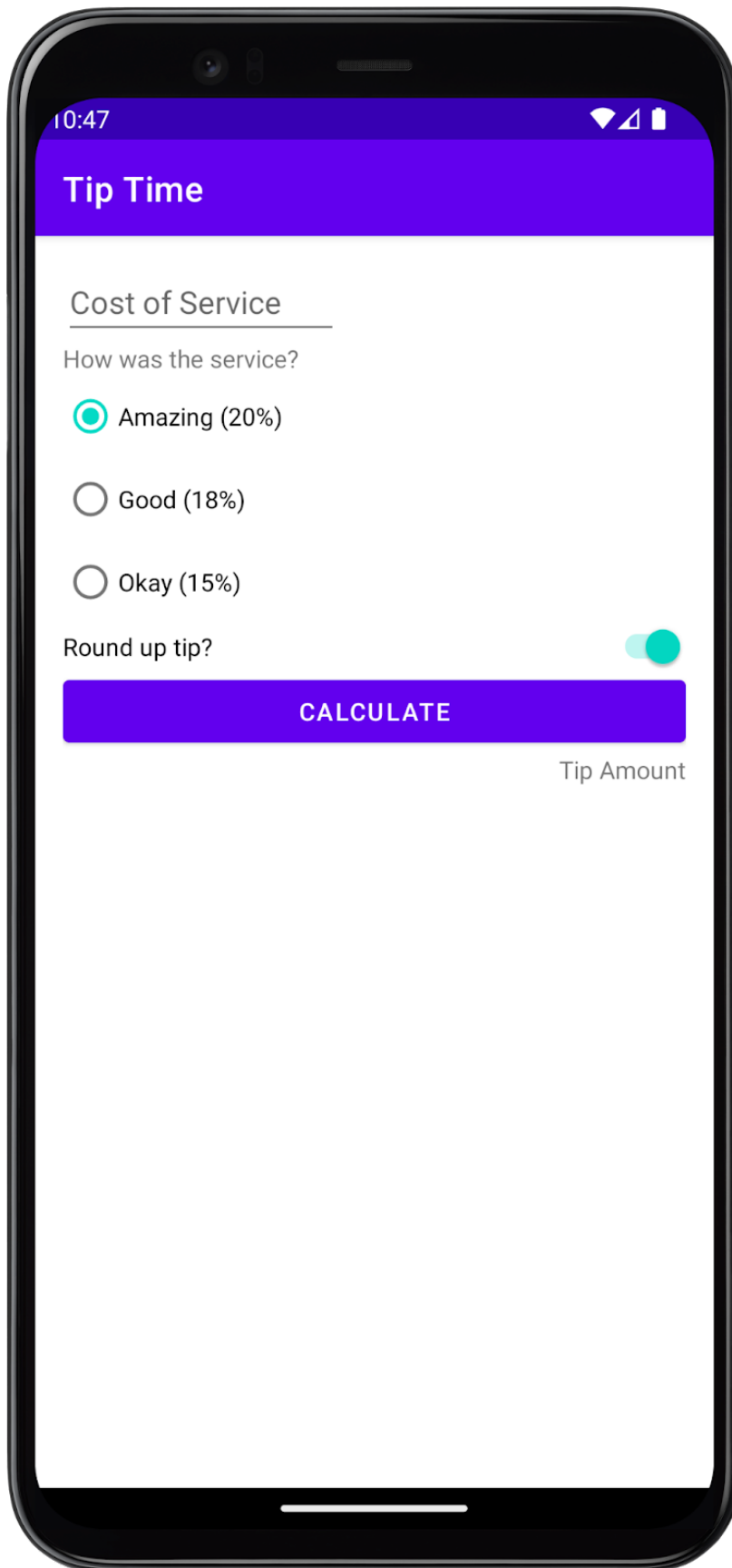
Reformat the XML

Android Studio provides various tools to tidy up your code and make sure it follows recommended coding conventions.

1. In `activity_main.xml`, choose **Edit > Select All**.
2. Choose **Code > Reformat Code**.

This will make sure the indenting is consistent, and it may reorder some of the XML of UI elements to group things, for example, putting all the `android:` attributes of one element together.

8. Solution code



10:47

Tip Time

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☒

CALCULATE

Tip Amount

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:padding="16dp"
tools:context=".MainActivity">

    <EditText
        android:id="@+id/cost_of_service"
        android:layout_width="160dp"
        android:layout_height="wrap_content"
        android:hint="@string/cost_of_service"
        android:inputType="numberDecimal"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/service_question"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/how_was_the_service"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/cost_of_service" />

    <RadioGroup
        android:id="@+id/tip_options"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checkedButton="@id/option_twenty_percent"
        android:orientation="vertical"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/service_question">

        <RadioButton
            android:id="@+id/option_twenty_percent"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/amazing_service" />

        <RadioButton
            android:id="@+id/option_eighteen_percent"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/good_service" />

        <RadioButton
            android:id="@+id/option_fifteen_percent"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/ok_service" />
    </RadioGroup>
```

```

<Switch
    android:id="@+id/round_up_switch"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:checked="true"
    android:text="@string/round_up_tip"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="@id/tip_options"
    app:layout_constraintTop_toBottomOf="@id/tip_options" />

<Button
    android:id="@+id/calculate_button"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="@string/calculate"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/round_up_switch" />

<TextView
    android:id="@+id/tip_result"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/tip_amount"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@id/calculate_button" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

res/values/strings.xml

```

<resources>
    <string name="app_name">Tip Time</string>
    <string name="cost_of_service">Cost of Service</string>
    <string name="how_was_the_service">How was the service?</string>
    <string name="amazing_service">Amazing (20%)</string>
    <string name="good_service">Good (18%)</string>
    <string name="ok_service">Okay (15%)</string>
    <string name="round_up_tip">Round up tip?</string>
    <string name="calculate">Calculate</string>
    <string name="tip_amount">Tip Amount</string>
</resources>

```

9. Summary

- XML (Extensible Markup Language) is a way of organizing text, made of tags, elements, and attributes.
- Use XML to define the layout of an Android app.
- Use `EditText` to let the user input or edit text.
- An `EditText` can have a hint to tell the user what is expected in that field.
- Specify the `android:inputType` attribute to limit what type of text the user can input into an `EditText` field.

- Make a list of exclusive options with `RadioButtons`, grouped with a `RadioGroup`.
- A `RadioGroup` can be vertical or horizontal, and you can specify which `RadioButton` should be selected initially.
- Use a `Switch` to let the user toggle between two options.
- You can add a label to a `Switch` without using a separate `TextView`.
- Each child of a `ConstraintLayout` needs to have vertical and horizontal constraints.
- Use "start" and "end" constraints to handle both Left to Right (LTR) and Right to Left (RTL) languages.
- Names of the constraint attributes follow the form `layout_constraint<Source>_to<Target>Of`.
- To make a `View` as wide as the `ConstraintLayout` it's in, constrain the start and end to the start and end of the parent, and set the width to `0dp`.

10. Learn more

Below are links to more documentation on the topics covered. You can find all of the [documentation for Android Development](#) on developer.android.com. And don't forget you can do a Google search if you get stuck on something.

- [TextView](#)
- [EditText](#)
- [Specify the input method type](#)
- [Radio Buttons guide](#)
- [RadioButton](#)
- [RadioGroup](#)
- [Switch](#)
- [ConstraintLayout](#)
- [XML](#) on Wikipedia

11. Practice on your own

Note: Practices are optional. They provide an opportunity for you to practice what you've learned in this codelab.

Do the following:

- Create a different calculator app, like a unit converter for cooking, to convert milliliters to or from fluid ounces, grams to or from cups, and so on. What fields do you need?

